Universidad de Buenos Aires
Facultad de Ingeniería
Año 2026 – 1.er Cuatrimestre

Taller de Automatización y Control

Trabajo Práctico n.° 1

Grupo 7

Lautaro García Vitale	110028	lagarciav@fi.uba.ar
Santiago Manuel Cabrera	110445	smcabrera@fi.uba.ar

Docentes: Dr. Ing. Claudio Pose – Ing. Tomás Gonzalez.

Resumen

En este trabajo se estudian y cuantifican las limitaciones de resolución y los retardos temporales introducidos por los sensores, actuadores y tareas auxiliares que intervienen en un sistema de control embebido basado en Arduino UNO R3. Se analizan un potenciómetro, un sensor ultrasónico, una IMU y un servomotor, determinando tanto sus resoluciones teóricas a partir de las hojas de datos como los tiempos reales de medición y actuación obtenidos experimentalmente. Además, se evalúa el impacto del envío de datos hacia una computadora en la caracterización del sistema. Este análisis permite disponer de una descripción cuantitativa precisa del desempeño del *hardware* utilizado, información esencial para el diseño y la correcta interpretación de futuros lazos de control.

Índice

1. Introducción **3**

2. Limitaciones de los sensores **3**
2.1. Potenciómetro 3
2.2. Sensor ultrasónico 4
2.3. IMU 4

3. Limitaciones de los actuadores **5**

4. Limitaciones de tareas adicionales **6**

5. Conclusiones **7**

6. Bibliografía **7**

A. Códigos referenciales **8**
A.1. Lectura del potenciómetro 8
A.2. Lectura del sensor ultrasónico 9
A.3. Lectura de la IMU 10
A.4. Lectura del servo 11
A.5. Envío de datos tipo `float` 12

1. Introducción

En el diseño de cualquier sistema de control embebido, el desempeño final no depende únicamente del algoritmo de control implementado, sino también de las limitaciones físicas y temporales impuestas por los sensores, los actuadores y las tareas auxiliares de adquisición y comunicación de datos. Estas limitaciones determinan la resolución con la que pueden medirse las variables de interés, así como los retardos que se introducen en el lazo de control, condicionando directamente la frecuencia máxima de operación y la calidad de la respuesta del sistema.

En el presente trabajo se analizan y cuantifican dichas limitaciones para el conjunto de dispositivos que serán utilizados a lo largo de la materia: un potenciómetro, un sensor ultrasónico, una unidad de medición inercial (IMU) y un servomotor, todos operados mediante una placa Arduino UNO R3. Para cada elemento se estudian tanto las resoluciones teóricas derivadas de sus especificaciones técnicas como los tiempos reales de medición y actuación obtenidos experimentalmente mediante código específico.

Asimismo, se evalúan los retardos adicionales introducidos por tareas complementarias, como el envío de datos hacia una computadora para su posterior procesamiento, los cuales, si bien no forman parte directa del lazo de control, influyen en la caracterización experimental del sistema.

El objetivo de este análisis es contar con una caracterización cuantitativa precisa de las capacidades y limitaciones del *hardware* disponible, información que resulta fundamental para el diseño adecuado de futuros lazos de control y para comprender los márgenes reales de desempeño alcanzables con la plataforma empleada.

2. Limitaciones de los sensores

En esta sección se detallan las limitaciones encontradas en los sensores utilizados, teniendo en cuenta las mínimas resoluciones y tiempos de medición de los mismos.

2.1. Potenciómetro

El conversor analógico-digital (ADC) del Arduino UNO R3 tiene una resolución de 10 bits, por lo que se dispondrá de $2^{10} = 1024$ valores tomados por el potenciómetro.

Se implementó la función `mapFloat()` que mapea un conjunto de valores de tipo flotante a otro conjunto del mismo tipo con un rango distinto —véase Apéndice A.1—. Teniendo en cuenta que el primer valor tomado por el potenciómetro es 0 y el último es 1023, la transformación lineal es

$$x^* = \frac{y_{\text{máx}} - y_{\text{mín}}}{1023} x + y_{\text{mín}} \,, \tag{1}$$

donde x es el valor que toma el potenciómetro y x^* es el valor mapeado al conjunto de rango $\{y_{\text{mín}}, y_{\text{máx}}\}$.

Se mapeo el valor $x = 1$ para una salida angular de entre 0° y 270°, obteniendo $\Delta\theta$, y luego para una salida de tensión de entre 0 V y 5 V, obteniendo ΔV como se muestra a continuación.

$$\Delta\theta = \frac{270^\circ}{1023} \approx 0,264^\circ$$

$$\Delta V = \frac{5\,\mathrm{V}}{1023} \approx 4,89\,\mathrm{mV}$$

En cuanto al tiempo requerido para realizar una medición, una conversión normal del ADC requiere 13 ciclos del reloj del ADC, mientras que la primera conversión luego de habilitar el módulo requiere 25 ciclos. En la placa, el microcontrolador opera a una frecuencia de $f_{\mathrm{CPU}} = 16\,\mathrm{MHz}$ y, por configuración por defecto, el ADC utiliza un *prescaler* de 128, por lo que la frecuencia del reloj del ADC resulta $f_{\mathrm{ADC}} = 16\,\mathrm{MHz}/128 = 125\,\mathrm{kHz}$, valor que se encuentra dentro del rango recomendado por el fabricante (50–200 kHz). En consecuencia, el tiempo de conversión normal es

$$t_{\mathrm{med}} = \frac{13}{f_{\mathrm{ADC}}} \approx 104\,\mu s,$$

mientras que el tiempo correspondiente a la primera conversión es $t^1_{\mathrm{med}} = \frac{25}{f_{\mathrm{ADC}}} \approx 200\,\mu s$.

Se midieron experimentalmente estos tiempos y se obtuvo

$$\widehat{t_{\mathrm{med}}} = (112 \pm 4)\,\mu\mathrm{s}$$

y $\widehat{t^1_{\mathrm{med}}} = 212\,\mu\mathrm{s}$, valores muy próximos a los teóricos.

2.2. Sensor ultrasónico

Según la hoja de datos del sensor, la máxima resolución en distancia es $\Delta x = 0,03\,\mathrm{m}$, por lo que la máxima resolución temporal es

$$\Delta t = 2\frac{\Delta x}{v_{\mathrm{sonido}}} = 17,48\,\mu\mathrm{s},$$

donde a 20 °C la velocidad del senido es de 343,2 m/s[2].

De este modo, teóricamente, el tiempo que tarda el sensor en medir una distancia de 50 cm es $t_{\mathrm{med}} = 2,91\,\mathrm{ms}$. Los resultados experimentales arrojaron un valor de $\widehat{t_{\mathrm{med}}} = 2,93\,\mathrm{ms}$, un valor muy cercano al teórico.

El código utilizado para este experimento se encuentra en el Apéndice A.2.

2.3. IMU

En esta sección se estimó la resolución del ángulo calculado a partir de las mediciones de la IMU, así como el tiempo que toma cada medición.

El ángulo se calcula a partir de las mediciones de aceleración según la Ecuación 2

$$\theta = \arctan\left(\frac{a_y}{a_z}\right), \tag{2}$$

donde a_y y a_z son las mediciones del acelerómetro. Además, según la hoja de datos de la IMU MPU6050 se utiliza un ADC de 16 bits para medir la aceleración. Si se trabaja en un rango de $\pm 8\,\mathrm{g}$ eso da una resolución de

$$\Delta a = \frac{16 \cdot g}{2^{16}} = 2,39\,\frac{\mathrm{mm}}{\mathrm{s}^2}. \tag{3}$$

Usando este resultado y propagando errores a partir de la Ecuación 2 se puede obtener la resolución de la estimación del ángulo.

$$\begin{aligned}
\Delta\theta &= \left|\frac{\partial\theta}{\partial a_y}\right| \cdot \Delta a_y + \left|\frac{\partial\theta}{\partial a_z}\right| \cdot \Delta a_z \\
&= \left|\frac{1}{1+(a_y/a_z)^2} \cdot \frac{1}{a_z}\right| \cdot \Delta a + \left|\frac{1}{1+(a_y/a_z)^2} \cdot \frac{-a_y}{a_z^2}\right| \cdot \Delta a \\
&= \left|\frac{1}{-9,8\,\mathrm{m/s^2}}\right| \cdot 2,39 \cdot 10^{-3}\,\mathrm{m/s^2} + 0 \\
&= 2,45 \cdot 10^{-4}\,\mathrm{rad} = 0,01^\circ
\end{aligned} \tag{4}$$

En esta ecuación se consideró que la IMU estaba en estado estático, de forma tal que $a_y = 0$ y $a_z = -9,8\,\mathrm{m/s^2}$. A pesar del resultado de la Ecuación 4, las mediciones de los acelerómetros son ruidosas, por lo que este cálculo tiene solo validez teórica.

Por otro lado, se estima el tiempo que se tarda en hacer una medición con la IMU. Según la hoja de datos, los sensores más lentos son los acelerómetros, los cuales son capaces de producir datos a una tasa de 1000 Hz, con lo que cada medición tardaría un tiempo $t_{\mathrm{med}} = 1\,\mathrm{ms}$.

Paralelamente, con el código que se muestra en el Apéndice A.3 se midió el tiempo que toma hacer una medición. El resultado fue $\widehat{t_{\mathrm{med}}} = 3048\,\mu\mathrm{s}$.

3. Limitaciones de los actuadores

Según la hoja de datos del servo, el mismo tiene una banda muerta de $2\,\mu\mathrm{s}$ a la hora de generar el ancho del pulso PWM vía Arduino, por lo que teóricamente la máxima resolución temporal es $\Delta t = 2\,\mu\mathrm{s}$. Este servomotor puede generar pulsos PWM de ancho 544-2400 μs, por lo que el tamaño del rango de valores que se le puede comandar al servo es

$$\text{rango} = \frac{2400\,\mu\mathrm{s} - 544\,\mu\mathrm{s}}{2\,\mu\mathrm{s}} = 928.$$

De esta manera, teniendo en cuenta que el ángulo variará entre -90° y 90°, teóricamente la máxima resolución angular es de

$$\Delta\theta = \frac{180^\circ}{928} = 0,19^\circ.$$

Asimismo, se comprobó experimentalmente que la máxima resolución temporal con la que se comanda la función `.writeMicroseconds()` es $\widehat{\Delta t} = (17 \pm 2)\,\mu$s, lo que corresponde a una máxima resolución angular de

$$\widehat{\Delta\theta} = (1,65 \pm 0,20)\,^{\circ}.$$

Por otro lado, según la hoja de datos, el servo tiene una velocidad de funcionamiento de 170 ms/60° a una tensión de 4,8 V, por lo que para un diferencial de 30° su velocidad de funcionamiento es de 85 ms/30°.

El código utilizado para este experimento se encuentra en el Apéndice A.4.

Adicionalmente, es posible estimar de manera experimental tanto la resolución angular efectiva como el tiempo que tarda el servo en desplazarse un diferencial de 30° utilizando una IMU externa solidaria al eje del servomotor. Para ello, pueden adquirirse los datos de orientación provistos por la IMU y enviarlos en tiempo real a una computadora, donde es factible visualizarlos y procesarlos, por ejemplo, mediante MATLAB. Sin embargo, este procedimiento introduce un nivel considerable de ruido debido a las mediciones inerciales, así como demoras adicionales asociadas a la adquisición, transmisión y procesamiento de los datos, que se suman al tiempo real de movimiento que se desea medir, afectando la precisión de la estimación.

Si la frecuencia de control fuese de 1 Hz, solo se modificarían los retardos, pero no las resoluciones.

4. Limitaciones de tareas adicionales

Por último, se calculó el tiempo que toma enviar 100 `floats` desde el Arduino hacia la computadora. Teniendo en cuenta que se trabaja a una tasa de envío de 115 200 bits por segundo y que cada `float` ocupa 32 bits, se tiene que

$$\text{tiempo de envío de un } \texttt{float} = \frac{32}{115\,200} = 277,8\,\mu\text{s}. \tag{5}$$

Asimismo, usando el código que se muestra en el Apéndice A.5 se midió el tiempo de envío. Cabe aclarar que la función `Serial.write()` no detiene el código hasta que los datos se envíen a la computadora, sino que los carga en un buffer. Para esperar hasta que los datos se envíen se usa la función `Seria.flush()`. Además, ya que se usa la función `micros()`, es necesario enviar varios `floats`, dado que la misma cuenta con una resolución de 4 µs. De esta manera y dividiendo el resultado por la cantidad de datos enviados se puede obtener un resultado más preciso. La medición se muestra en la Ecuación 6.

$$\text{tiempo estimado de envío de un } \texttt{float} = 340,1\,\mu\text{s} \tag{6}$$

Finalmente, hay que mencionar que a la hora de enviar datos a MATLAB se usa un encabezado de 4 *bytes*, lo que equivale a enviar un `float` más.

5. Conclusiones

Se estimaron y midieron las resoluciones de los actuadores y sensores que se utilizarán en el desarrollo de la materia. Los resultados se resumen en la Tabla 1.

Dispositivo	Valor teórico	Valor estimado	Unidad
Potenciómetro	0,264 | 4,89	-	° | mV
Sensor ultrasónico	0,03 | 17,48	-	m | μs
IMU	2,39 | 0,01	-	mm/s^2 | °
Servo	2 | 0,19	17 | 1,65±0,20	μs | °

Tabla 1: Resumen de resoluciones.

Se estimaron y midieron los retardos que añaden los actuadores y sensores, lo que resulta fundamental a la hora de diseñar un lazo de control. Los resultados se resumen en la Tabla 2.

Tarea	Valor teórico	Valor estimado	Unidad
Lectura potenciómetro	200 | 104	212 | 112 ± 4	μs
Lectura sensor ultrasónico	2,91	2,93	ms
Lectura IMU	1000	3048	μs
Comando servo	85	-	ms/30°
Envío de datos	277,8	340,1	μs/`float`

Tabla 2: Resumen de retardos.

6. Bibliografía

[1] Morgan, Elijah J., *HC-SR04 Ultrasonic Sensor*, 2014

[2] InvenSens Inc, *MPU 6000 and 6050 Product Specification Revision 3.3.*, 2012.

[3] Handson Technology, *MG996R Metal Gear Servo Motor.*

[4] Arduino, *Arduino UNO R3 Product Reference Manual*, Rev. 3, 30/04/2024.

[5] Microchip Technology, *ATmega328P Datasheet – 8-bit AVR Microcontroller*, Complete Datasheet.

Apéndices

A. Códigos referenciales

A.1. Lectura del potenciómetro

```
// El potenciómetro tiene algo de ruido (a veces glitchea)

void setup() {
  Serial.begin(115200); // Inicia la comunicación serial
}

float mapFloat(float valor, float x_inicial, float x_final, float y_inicial, float y_final){
  return (y_final - y_inicial)/(x_final - x_inicial) * (valor - x_inicial) + y_inicial;
}

void loop() {

  // Resolución

  /*int valor = analogRead(A0);

  float angulo = mapFloat((float)valor, 0, 1023, 0, 270);
  float tension = 1000 * mapFloat((float)valor, 0, 1023, 0, 5); //Se multiplica por 1000 para ver mejor en el monitor
  Serial.println(tension);*/

  // Tiempo de medición

  unsigned long t_ini = micros();

  int valor = analogRead(A0);

  unsigned long t_fin = micros();

  Serial.println((t_fin - t_ini));
}
```

A.2. Lectura del sensor ultrasónico

```
#include <NewPing.h>

#define TRIGGER_PIN  6   // Arduino pin tied to trigger pin on the ultrasonic sensor.
#define ECHO_PIN     7   // Arduino pin tied to echo pin on the ultrasonic sensor.
#define MAX_DISTANCE 100 // Maximum distance we want to ping for (in centimeters).
↪  Maximum sensor distance is rated at 400-500cm.

NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE); // NewPing setup of pins and
↪  maximum distance.

void setup() {
  Serial.begin(115200);
}

void loop() {
  unsigned long t_ini = micros();
  unsigned long med = sonar.ping(MAX_DISTANCE);
  unsigned long t_fin = micros();
  Serial.println(med);

  while(micros() - t_ini < 1000000){}
}
```

A.3. Lectura de la IMU

```
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <Wire.h>

Adafruit_MPU6050 mpu;

void setup() {
  Serial.begin(115200);
  while (!Serial)
    delay(10);

  Serial.println("Adafruit MPU6050 test!");

  // Try to initialize!
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip");
    while (1) {
      delay(10);
    }
  }
  Serial.println("MPU6050 Found!");

  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_44_HZ);


}

void loop() {
  unsigned long t_ini = micros();

  sensors_event_t a, g, t;
  mpu.getEvent(&a, &g, &t);
  unsigned long t_fin = micros();

  Serial.println(t_fin - t_ini);

  while (micros() - t_ini < 1000000) {}
}
```

A.4. Lectura del servo

```
#include <Servo.h>

Servo miServo;

void setup() {
  miServo.attach(9); // pin PWM
  Serial.begin(9600); // Inicia la comunicación serial
}

void loop() {

  /*for(int i = 0; i < 20; i++){
    miServo.writeMicroseconds(1500 + 1*i);
    Serial.print(i);
    Serial.print(":");
    Serial.println(miServo.read());
    delay(250);
  }*/

  miServo.write(90);
  delay(1000);
  unsigned long t_ini = micros();
  miServo.write(120);
  while(miServo.read() != 120){}
  unsigned long t_fin = micros();

  Serial.println(t_fin - t_ini);
  delay(1000);
  //while (micros() - t_ini < 20000) {}
}
```

A.5. Envío de datos tipo `float`

```
#define N 100

void setup() {
  Serial.begin(115200);

}

void loop() {
  float dato = 4.5;
  Serial.flush(); //Vacía el buffer

  unsigned long t_ini = micros();

  for(int i = 0; i < N; i++){
    matlab_send(&dato, 1);
  }

  Serial.flush(); //Espera que el buffer se vacíe.
  unsigned long t_fin = micros();

  Serial.println("");
  Serial.print("En envio tardo: ");
  Serial.println((float)(t_fin - t_ini)/N);

  while(micros() - t_ini < 500000){}
}


void matlab_send(float *datos, size_t largo){
  for(size_t i = 0; i < largo; i++){
    byte * b = (byte *) &datos[i];
    Serial.write(b,4);
  }
}
```